Michelle T. George

Python – Intermediate Activity Handouts

Activity I1 – Problem Decomposition with Functions

Title: Breaking a Bigger Problem into Small Functions

Context

You now know variables, loops, if/else, and simple functions.
Next: learn to split a problem into clear sub-tasks.

Problem

Given a list of student marks, you must:

- Drop the lowest mark.
- Compute the new average.
- Decide pass/fail using a threshold.

Part A – Think in parts

1. Write the overall problem in one sentence:
   To compute new average, lowest marks, deciding, pass/fail.
2. Identify at least three smaller tasks:
   - Task 1: lowest mark
   - Task 2: new average
   - Task 3: deciding pass/fail.
3. For each task, write a function name and its input and output:
   - Function 1 name: def drop_lowest(marks):
     - Input(s): list [20, 25, 30]
     - Output: 20
   - Function 2 name: def average(marks):
     - Input(s): list [20, 25, 30] / 3
     - Output: 25
   - Function 3 name: def is_pass
     - Input(s): 80% / 40%
     - Output: pass fail / true / false

Part B – Implement functions

Fill and complete:

python

```
def drop_lowest(marks):
    # TODO: remove the smallest mark and return the new list
    return marks output ; 20


def average(marks):
    total = 0
```

```
for m in marks:
    total = total + m
return total / len(marks)

def is_pass(avg, threshold):
    # return True or False
    if >avg :
        return True
    else: ≤ avg
        return False

# Main flow
marks_list = [20, 25, 30, 35]
threshold = 27.5

new_marks = drop_lowest(marks_list)
avg = average(new_marks)
result = is_pass(avg, threshold)
print("Marks after dropping lowest:", new_marks)
print("Average:", avg)
print("Pass?", result)
```

```
def drop_lowest(marks):
    if marks:
        marks.remove(min(marks))
    return marks
def average(marks):
    if not marks: return 0
    total = 0
    for m in marks:
        total = total + m
    return total / len(marks)
```

Reflection

- How did dividing into functions help you think more clearly?

easy to understand

Activity I2 – Working with Dictionaries

Title: Simple Student Record System

Problem

You need to manage student data: name, roll number, and marks.
You will use dictionaries for this.

Part A – Design

1. Decide a structure for one student record using a dict:

python

```
student = {
  "roll": 37,
  "name": "Michelle",
  "marks": [70, 60, 80]
}
```

2. What keys will you use?

- Key 1: append (for adding)
- Key 2: split (for splitting)
- Key 3: len (for counting

Part B – Code

Complete:

python

```
students = []

# Add 3 students
for i in range(3):
  roll = int(input("Enter roll: "))
  name = input("Enter name: ")
  m1 = int(input("Mark 1: "))
  m2 = int(input("Mark 2: "))
  m3 = int(input("Mark 3: "))
  record = {
    "roll": roll,
    "name": name,
    "marks": [m1, m2, m3]
  }
  students.append(record)
```

```
# Print each student name and average mark
for s in students:
    marks = s["marks"]
    avg = sum(marks) / len(marks)
    print(s["roll"], s["name"], "Average:", avg)
```

Extension (optional)

- Add a decision: print "Pass" if average ≥ 50 else "Fail".

```
if average ≥ 50:
    print ("pass")
else:
    print ("fail")
```

Activity 13 – Reading and Writing Files

Title: Process Data from a Text File

Problem

You are given a text file marks.txt with one mark per line.
You must read it and write a summary to summary.txt.

Part A – Understand

1. Inputs (file/content): marks.txt
2. Outputs (file/content): Summary.txt

Part B – Code Skeleton

python

```
# Step 1: Read marks from file
marks = []
with open("marks.txt", "r") as f:
    for line in f:
        line = line.strip()
        if line != "":
            marks.append(int(line))
# Step 2: Compute statistics
total = sum(marks)
count = len(marks)
average = total / count
max_mark = max(marks)
count = len([m for m in mark if m>= 60])
min_mark = min(marks)
# Step 3: Write summary to new file
with open("summary.txt", "w") as f:
    f.write("Number of students: " + str(count) + "\n")
    f.write("Average mark: " + str(average) + "\n")
    f.write("Max mark: " + str(max_mark) + "\n")
    f.write("Min mark: " + str(min_mark) + "\n")
f.write("mark ≥ 60:" + str(count 60) + "\n")
```

Tasks

- Add one more line to summary: count of students with mark ≥ 60.
- Write that line in the file.

Reflection

- How is processing from file different from input()?

diffrent for using file.

Activity 14 – Using Libraries (math / statistics)

Title: Use Standard Library for Cleaner Code

Problem

You need to analyze a list of numbers with mean, median, and standard deviation.

Part A – Manual vs Library

1. In simple words, what do these mean:
   - mean: average of total
   - median: middle value when data is sorted
2. Write plain Python code (no extra library) to compute mean.

python

```
numbers = [10, 20, 30, 40, 50]
# Your mean logic here
```

Part B – Use statistics module

python

```
import statistics as stats

numbers = [10, 20, 30, 40, 50]
mean_value = stats.mean(numbers)
median_value = stats.median(numbers)
stdev_value = stats.pstdev(numbers)

print("Mean:", mean_value)
print("Median:", median_value)
print("Std Dev:", stdev_value)
```

Task

- Add code to ask the user for numbers (comma-separated) and then compute these three values using the library.

```
import statistics as stats
input ("Enter numbers separated by comma.)
numbers = [float (x) for x in user input.split (",")]

mean_value = stats.mean(numbers
median_value = stats.median (numbers)
Std dev_value = stats.pstdev (numbers )

print ("mean*:", mean_value]
print ("median:", median_value]
print ("stddev:", stdev_value
```

of numbers
10, 20, 30, 40, 50

Mean 30

Median 30

Std dev: 14.142135...

Activity I5 – Problem Template with Complexity Awareness

Title: Compare Two Approaches (Efficiency)

Problem
You must check if a number is present in a very large list (e.g., 1 million items).

Part A – Two Approaches

Approach 1: Simple loop search.
Approach 2: Use a set.

1. Describe Approach 1 in words:
Check every item in the list one by one using a loop until the target number is found or the list ends
2. Describe Approach 2 in words:
Convert the list to a set first, then check if the target is in the set.

Part B – Implement Both

python

```
import time


# Create large list

nums = list(range(1000000))

target = 999999


# Approach 1: List search

start = time.time()

found = False

for x in nums:

    if x == target:

        found = True

        break

end = time.time()

print("List search found:", found, "Time:", end - start)


# Approach 2: Set search

num_set = set(nums)

start = time.time()

found2 = (target in num_set)

end = time.time()

print("Set search found:", found2, "Time:", end - start)
```

Reflection

- Which was faster and why (in simple words)?

Using set was faster because it is like an index that lets python jump straight to the number to check if it exist It doesn't need to look at every item so it is easy.

Activity I6 – Input Validation and Error Handling

Title: Robust User Input

Problem

You want a safe function that always returns a valid integer entered by the user.

Part A – Requirements

1. What should happen if user types "abc"?

It will print invalid input. Because abc is not an integer so value error will occur.

2. What should happen if user leaves it blank?

An empty input cannot be converted to int, so it raises value error so program will give invalid input.

Part B – Code with try/except

python

```
def get_int(prompt):
    while True:
        user_input = input(prompt)
        try:
            value = int(user_input)
            return value
        except ValueError:
            print("Invalid input. Please enter an integer.")

age = get_int("Enter your age: ")
print("Your age is:", age)
```

abc

Tasks

- Modify get_int to also enforce a range (e.g., 0 to 120).
- If user enters out-of-range value, show a message and ask again.

```
def get_int(prompt, min_val = 0, max_val = 120):
    while True:
        user_input = input(prompt)
        try:
            value = int(user_input)
            if min_val <= value <= max_val:
                return value
            else:
                print(f"out of range. please enter a)
```

Activity I7 – Mini Project: Text Analyzer

Title: Word Frequency Counter

Problem
Given a short paragraph, count how many times each word appears.

Part A – Plan

Use the problem template (short version):

1. Inputs: A sentence or paragraph entered by user as a string
2. Outputs: It will count how many times a word appears
3. Steps (bullet points):
   - Get text input from the user
   - Convert all text to lowercase so "The" and "the" count as same word
   - Split text into individual words
   - Use a dictionary key to keep count of each word
   - loop through all words and update the count in dictionary
   - print each word with its frequency

Part B – Implement

python

```
text = input("Enter a sentence/paragraph: ")


# Convert to lower case

text = text.lower()


# Split into words

words = text.split()


# Count frequencies using dictionary

freq = {}

for w in words:

  if w in freq:

    freq[w] += 1

  else:

    freq[w] = 1


# Print results

for word, count in freq.items():

  print(word, ":", count)
```

Extensions (optional)

- Remove punctuation before counting.

Activity 18 – Applying the Full Problem-Solving Workflow

Title: Generic Approach Practice – Loan EMI Calculator

Problem

Compute the Equated Monthly Instalment (EMI) for a loan given:

- Principal (P)
- Annual interest rate (R, in %)
- Tenure in years (N)

Formula (you may give to students):

$$EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$$

where

- $r = R/(12 \times 100)$ (monthly rate)
- $n = N \times 12$ (months)

Use Full Template

1. Problem understanding
   - Inputs: Principal (P) - loan amt, Annual interest rate (R) in % Tenure in years (N)
   - Outputs: EMI amount per month
   - Constraints: P, R, N - +ve number
     R is annual rate, must convert to monthly rate.
     N is in years, must convert to months.
2. Analysis (steps)
   - Convert annual interest rate to monthly rate
   - ~~R is annual rate convert to monthly rate~~
   - N is in years to months. n = N×12
   - Apply EMI formula.
3. Design
   - Functions: e.g., compute_emi(P, R, N)? Yes / No
   - Data types: float/int? ~~float for R~~, R, EMI and r. int for N, n
4. Code

python

```
def compute_emi(P, R, N):
    r = R/(12*100)
    n = N*12
    emi = P/n
    return emi


P = float(input("Principal: "))
R = float(input("Annual interest rate (%): "))
N = int(input("Tenure (years): "))
```